1

a) $0.000045678 10$

$= 4.56781 \times 10^{-5}$

b) 6789.345×10^{4}

$= (6.789345)_{10} \times 10^{7}$

c) 0.0011011

$\cancel{= 1.1011_2 \times 10^{-7}}$

$= (1.1011)_2 \times 2^{-3}$

d) 101101.11011101101_2

$= 1.0110111011101101_2 \times 2^{5}$

~~e)~~

★ $1.1101_2 \times 2^{14}$ find biased exponent

Bias = 127

Biased exponent $= 127 + 14 = 141$

$= (10001101)_2$

★ $1.0011_2 \times 2^{-5}$

Bias $= 2^{5-1} - 1 = 15$

Biased exponent $= 15 + (-5) = 10$

$= (01010)_2$

Q.2 Convert 0.003921$_{10}$ into 32 bit IEEE-754

Sign bit = 1

Exponent = 127 − 8 = 119

= (01110111)

converting into binary,

$(0.003921)_{10} = (0.000\,0000\,0100000000111)_2$

Normalized = 1.00000000×2^{8}

∴ Mantissa = 00000000000000000000000

∴ Ans = 1, 01110111 00000000000000000000000

(2) 1101.1110×2^{53}

$= 1.1011110 \times 2^{56}$

Exponent = 56 + 1023 = 1079

$= (10000110111)_2$

Sign bit = 0

∴ mantissa = 1011110000000000000000000000000

0000000000000000000000

(1) 0xC1200000

Binary: 1100 0001 $\overset{0010}{\cancel{000}}$ 0000 0000 0000 0000 0000

A 1010
B 1011
C 1100

∴ sign bit = 1
Exponent = 10000010 = 130 ⇒ 130 − 127 = 3

Value = $-1.01_2 \times 2^3$
= −10 (Ans)

(2) 0x4059000000000000

Binary: 0100 ~~0101~~ 0000 $\overset{0101}{\cancel{0100}}$ 1001
0000 0000 0000 0000 0000 0000
0000 0000 0000 0000 0000 0000

Sign = 0 (pos)
Exp = 10000000101 = 1029
∴ 1029 − 1023 = 6

Value = $1.1001_2 \times 2^6$
= $(100.0)_{10}$

2

$0.000101_2 \times 2^{-120}$ | $101.001_2 \times 2^{100}$

$= 1.01 \times 2^{-124}$ | $= 1.01001 \times 2^{102}$

Therefore,

$1.01 \times 2^{-124} \times 1.01001 \times 2^{102}$

$= (1.01 \times 1.01001) \times 2^{-124+102}$

$= 1.1001101 \times 2^{-22}$

No overflow as -22 is in range.

3

1.011×2^{-120} | 1111.0111×2^{67}

$= 1.1110111 \times 2^{70}$

Therefore,

$1.011 \times 2^{-120} \times 1.1110111 \times 2^{70}$

$= (1.011 \times 1.1110111) \times 2^{-120+70}$

$= 1.0101011101 \times 2^{-50}$

actual exponents $= -50$

biased " $= -50 + 1023 = 973$ [in range]

$\therefore$ no overflow

CamScanner

(3)

0.0101×2^{790}
$= 1.01 \times 2^{788}$

1010.0101×2^{680}
$= 1.0100101 \times 2^{683}$

18 bit
1 sign
5 ~~frac~~ expo
12 ~~expo~~ frac

$1.01 \times 2^{788} \times 1.0100101 \times 2^{683}$

$= (1.01 \times 1.0100101) \times 2^{683+788}$

$= (1.1000110) \times 2^{1471}$

1471 is not in range.

∴ Overflow.

4) Add 7.01127 and 10.11120 in IEEE

7.01127 in binary
$= 111.000000111\ldots_2$
$= 1.11000000111 \times 2^2$

10.11120 in binary
$= 1010.000111000111$
$= 1.010000111 0 \times 2^3$

$1.11000000111 \times 2^2 + 1.0100001110 \times 2^3$

$\Rightarrow 0.111000000111 \times 2^3 + 1.0100001110 \times 2^3$

$\Rightarrow 2^3(10.00100100\ldots)$

$= 1.0001001000 \times 2^4$

as $4+127=131$ which is within range.

$\therefore$ There is no overflow/underflow.

4) Sub 18.11218 from 101.00121

101.00121	18.11218
$= 1100101.000000_1\ldots$	$= 10010.000111001$
$= 1.1001010000 \times 2^6$	$= 1.0010000111001 \times 2^4$

Now,

$1.1001010000 \times 2^6 - 1.0010000111001 \times 2^4$

$= 1.1001010000 \times 2^6 - \cancel{1.001}\ 0.010010000111001 \times 2^6$

$= 2^6(1.1001010000 - 0.010010000111001)$

$= 1.0100101110001110000 \times 2^6$

no overflow, underflow as $6+127=133$ is within range

5

1) Normalization is necessary because it ensure a unique numerical number representation for every number. ~~Besides~~, For example:

0.1×10^{-1} and 0.01×10^{0} represent the same value without normalization. but with normalisation, 11)

Overflow	Underflow
Overflow means the value is too large to be represented in the given exponent range.	Underflow means the value is too small to be representated in the given exponent range.
<u>for example</u> in single precision 10^{500} will result in a overflow.	<u>for example</u> 10^{-400} will result in a underflow.

iii)

Purpose of bias

Bias allows storing positive and negative exponents using a seperate sign bit.

6.

fadd.s rd, rs1, rs2

Working principle

1. reads two single precision float values ~~and store~~ from registers rs1 & rs2.

2. Performs IEEE-754 addition and then writes the value into rd.

fmsub.s rd, rs1, rs2, rs3

Working principle

Performs $(rs_1 \times rs_2) - rs_3$ in a single precision.

Then, the result is stored in rd.

flw → flw rd, offset(base)

Working principle

loads a single precision floating point value from ~~register~~ memory at offset(base) to floating point register rd.

feq.s → feq.s, rd, rs1, rs2

Working principle

Compares the single precision floating values in registers rs1 and rs2.

If (rs1==rs2) : rd = 1 else rd = 0

fcvt.s.w → fcvt.s.w rd, rs1

Working principle

Converts a 32 bit signed integer from register rs1 into a single precision (IEEE-754) and stores result in rd.